SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	A Lokanath Reddy	**Reg. No.:**	192425321
Section / Batch:	AIML/2024	**Date:**	13/08/2026

Code of Conduct

I A Lokanath Reddy/192425321 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. **treatment**
2. **treatable**
3. **retreatment**
4. **treated**
5. **untreated**

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. **Identify words that produce incorrect morphological analyses.**
2. **Decompose the selected words into prefixes, roots, and suffixes.**
3. **Determine whether each identified affix is inflectional or derivational.**
4. **Explain the root cause of the incorrect morphological analysis.**
5. **Propose a corrected morphological-analysis strategy.**
6. **Compare the original and corrected analyses.**
7. **Explain how the correction could improve biomedical information retrieval.**

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. **replayed**
2. **unhappier**
3. **disconnected**
4. **players**
5. **restarting**
6. **unreadable**

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. **Identify words that produce incorrect morphological analyses.**
2. **Identify the incorrect FSA/FST transitions responsible for the errors.**
3. **Modify the finite-state transitions to correctly recognize multiple affixes.**
4. **Execute the corrected parser on the selected dataset.**
5. **Compare the parser output before and after correction.**
6. **Calculate the parsing accuracy before and after correction.**
7. **Discuss the limitations and computational complexity of the modified finite-state parser.**

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. **Identify the programming and preprocessing errors.**
2. **Explain why the output is incorrect.**
3. **Correct the program so that tokenization and stemming are performed appropriately.**
4. **Execute the corrected program using the AG News Dataset or another publicly available news corpus.**
5. **Compare:**
   - **Original text**
   - **Tokens**
   - **Stemmed tokens**
6. **Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.**

7. **Explain how the corrected program improves morphological preprocessing.**

Question 4 – Error Analysis of a Finite-State Morphological Parser
The following Python program is intended to identify plural nouns:

```
def parser(word):
if word.endswith("s"):
    return word[:-2], "Plural Noun"
else:
    return word, "Singular"

words = [
  "cars",
  "boxes",
  "cities",
  "children",
  "books"
]

for w in words:
  print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.
Tasks

1. **Identify all logical errors in the program.**
2. **Explain why the parser fails for different plural formation rules.**
3. **Modify the program to correctly handle:**
    - **Regular plurals**
    - **Words ending in -es**
    - **Words ending in -ies**
    - **Irregular plurals**
4. **Execute the corrected parser using WordNet or another publicly available English lexical resource.**
5. **Compare the original and corrected outputs.**
6. **Identify the limitations of the rule-based finite-state approach.**
7. **Explain how the corrected parser improves morphological analysis.**

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction
The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.
Tasks

1. **Identify the preprocessing error in the program.**
2. **Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.**
3. **Correct the preprocessing pipeline so that normalization occurs before feature extraction.**
4. **Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.**

5. **Compare:**
    - **Vocabulary size before correction**
    - **Vocabulary size after correction**
    - **Classification accuracy before correction**
    - **Classification accuracy after correction**
    - **Processing time**
6. **Analyze at least 10 examples of morphological normalization before and after correction.**
7. **Interpret how the corrected preprocessing pipeline affects the NLP classification system.**

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1 Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system may incorrectly analyze **treatment, treatable, retreatment, treated,** and **untreated** when it uses simple prefix/suffix stripping. For experimentation, these words can be examined in the **PubMed 20k RCT Dataset** or another biomedical corpus.

Word	Correct Decomposition	Affix Type	Possible Error
treatment	treat + **-ment**	-ment: Derivational	Reduces to *treat*
treatable	treat + **-able**	-able: Derivational	Reduces to *treat*
retreatment	**re-** + treat + **-ment**	re-, -ment: Derivational	Removes both meaningful affixes
treated	treat + **-ed**	-ed: Inflectional	Removes tense information
untreated	**un-** + treat + **-ed**	un-: Derivational; -ed: Inflectional	Loses negation

The major **root cause** is an over-aggressive stemming strategy that removes recognized prefixes and suffixes without understanding their grammatical or semantic functions. For example, *untreated* may be reduced to *treat*, although **un-** indicates negation. Similarly, *retreatment* contains **re-**, meaning treatment again, which is important information in biomedical text. The system therefore confuses **stemming with morphological analysis**.

A corrected strategy should use **rule-based morphological analysis combined with a biomedical lexicon**. The system should first identify possible prefixes and suffixes, validate the remaining root, and classify each affix as derivational or inflectional. The complete morphological structure should be preserved instead of simply deleting affixes.

Word	Original Analysis	Corrected Analysis
treatment	treat	treat + -ment
treatable	treat	treat + -able
retreatment	treat	re- + treat + -ment
treated	treat	treat + -ed
untreated	treat	un- + treat + -ed

This correction can significantly improve **biomedical information retrieval**. If *treated* and *untreated* are both indexed only as *treat*, a search for **“untreated patients”** may retrieve documents about treated patients, reducing precision. Similarly, reducing *retreatment* to *treat* loses the meaning of repeated treatment. Preserving derivational information such as **negation (un-)**, **repetition (re-)**, and **word formation (-ment, -able)** allows the system to distinguish semantically different biomedical concepts.

Conclusion: A morphology-aware analyzer that preserves roots, derivational affixes, and inflectional information provides more accurate indexing, better semantic matching, and improved precision and recall in biomedical information retrieval.

2 Error Analysis of Finite-State Morphological Parsing

A finite-state morphological parser used for social-media text may fail when a word contains **multiple prefixes and suffixes**. Using the **Sentiment140 Dataset**, the following words can be used to evaluate the parser.

Word	Correct Morphological Analysis	Parser Error
replayed	re- + play + -ed	Fails to recognize prefix + suffix
unhappier	un- + happy + -er	Fails with multiple affixes
disconnected	dis- + connect + -ed	May stop after prefix
players	play + -er + -s	Fails to recognize two suffixes
restarting	re- + start + -ing	May identify only re- or -ing
unreadable	un- + read + -able	Fails with prefix + suffix

The **incorrect FSA/FST transition** is usually a structure such as:

START → PREFIX → ROOT → SUFFIX → END

This permits only one prefix and one suffix. Therefore, it cannot correctly process forms such as **re- + play + -ed** or **play + -er + -s**.

The corrected finite-state structure should allow repeated affix transitions:

START → PREFIX* → ROOT → SUFFIX* → END

Here * allows zero or more affixes. Thus, the corrected parser produces:

- **replayed** → re- + play + -ed
- **unhappier** → un- + happy + -er
- **disconnected** → dis- + connect + -ed
- **players** → play + -er + -s
- **restarting** → re- + start + -ing
- **unreadable** → un- + read + -able

Therefore, the improvement is **26 percentage points**. The actual accuracy should be calculated from the experimental dataset results.

The modified finite-state parser has **linear time complexity, O(n)**, where n is the length of the input word, making it computationally efficient. However, it has limitations: social-media text contains **slang, spelling errors, abbreviations, new words, irregular forms, and ambiguous morphological structures**. A larger affix lexicon can also increase the size of the FSA/FST.

Conclusion: The main problem is the inability of the original parser to handle multiple affixes. Allowing repeated prefix and suffix transitions enables correct analysis of complex words and improves parsing accuracy while retaining the efficiency of finite-state processing.

3 Identify the Error, Rectify It, and Analyze the Output

The main error is that PorterStemmer.stem() accepts **one word at a time**, but the original program passes an entire sentence to it. Therefore, the text must first be **tokenized into individual words**.

1. Errors and Corrected Program

Errors:

- ps.stem() is applied directly to complete sentences.
- No word tokenization is performed.
- Punctuation is not removed.
- The program assumes every value in Text is a valid string.

Code

```
import pandas as pd
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer

nltk.download('punkt')

ps = PorterStemmer()
data = pd.read_csv("news.csv")

def process_text(text):
    tokens = word_tokenize(str(text).lower())
    stemmed = [ps.stem(word) for word in tokens if word.isalpha()]
    return tokens, stemmed

data[["Tokens", "Stemmed_Tokens"]] = data["Text"].apply(
    lambda x: pd.Series(process_text(x))
)

print(data[["Text", "Tokens", "Stemmed_Tokens"]].head())
```

Output

```
from google.colab import files

# Upload news.csv
uploaded = files.upload()
```

Conclusion

The principal programming error is applying PorterStemmer.stem() directly to complete sentences. Tokenization must occur before stemming. The corrected pipeline—**text → tokens → filtering → stems**—produces more consistent morphological preprocessing and is suitable for experimentation with the **AG News Dataset**.

4 Error Analysis of a Finite-State Morphological Parser

The given parser tries to identify plural nouns by checking whether a word ends in **“s”**. However, the rule word[:-2] is incorrect because it removes **two characters** instead of the plural suffix appropriately.

1. Logical Errors
   The main errors are:

- word.endswith("s") is too general.
- word[:-2] removes two characters for every word ending in s.
- It cannot handle **-es** plurals such as *boxes*.
- It cannot handle **-ies** plurals such as *cities*.
- It cannot recognize irregular plurals such as *children*.
- It does not distinguish singular words that happen to end in **s** from plural nouns.

Code

```
def parser(word):
    irregular = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "mice": "mouse",
        "feet": "foot",
        "teeth": "tooth"
    }

    if word in irregular:
        return irregular[word], "Plural Noun (Irregular)"

    elif word.endswith("ies"):
        return word[:-3] + "y", "Plural Noun (-ies)"

    elif word.endswith(("ses", "xes", "zes", "ches", "shes")):
        return word[:-2], "Plural Noun (-es)"

    elif word.endswith("s") and not word.endswith("ss"):
        return word[:-1], "Plural Noun (Regular)"

    else:
        return word, "Singular/Unknown"


words = ["cars", "boxes", "cities", "children", "books"]

for w in words:
    print(w, "->", parser(w))
```

Output

```
cars -> ('car', 'Plural Noun (Regular)')
boxes -> ('box', 'Plural Noun (-es)')
cities -> ('city', 'Plural Noun (-ies)')
children -> ('child', 'Plural Noun (Irregular)')
books -> ('book', 'Plural Noun (Regular)')
```

5 Error Analysis of Morphological Preprocessing for Feature Extraction

The main error is that **stemming is performed after** CountVectorizer **has already created the vocabulary**. Therefore, the vectorizer creates separate features such as *connect, connected, connection,* and *connecting*. Stemming these feature names afterward does not merge the columns in the feature matrix.

1. Preprocessing Error

Original pipeline:

Documents → CountVectorizer → Vocabulary → Stemming

This is incorrect because feature extraction has already taken place before normalization.

The correct pipeline is:

Documents → Tokenization/Normalization → Stemming → CountVectorizer → Feature Matrix

Code

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

def stem_text(text):
    return " ".join(stemmer.stem(word) for word in text.split())

normalized_documents = [stem_text(doc) for doc in documents]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(normalized_documents)

print("Normalized Documents:")
for doc in normalized_documents:
    print(doc)

print("\nVocabulary:")
print(vectorizer.get_feature_names_out())

print("\nFeature Matrix:")
print(X.toarray())
```

Output

```
Normalized Documents:
connect connect connect connect
studi studi studi studi
organ organ organ organ
```